Adrián Varela Jiménez, Diego Román Sánchez,
Adrián García García, Hugo Zafra Alarcos

PRÁCTICA 4 ALGORÍTMICA GRUPO A3 LIANA



UNIVERSIDAD
DE GRANADA

Índice

1. Selección de proyectos
    1.1. Backtracking
        1.1.1. Descripción de la solución
        1.1.2. Análisis de eficiencia
        1.1.3. Ejecución
    1.2. Branch & Bound
        1.2.1. Descripción de la solución
        1.2.2. Análisis de eficiencia
        1.2.3. Ejecución
    1.3. Comparativa
2. Coloreo de grafos
    2.1. Backtracking
        2.1.1. Descripción de la solución
        2.1.2. Análisis de eficiencia
        2.1.3. Ejecución
    2.2. Branch & Bound
        2.2.1. Descripción de la solución
        2.2.2. Análisis de eficiencia
        2.2.3. Ejecución
    2.3. Comparativa
        2.3.1 Ejecución map_coloring_example
        2.3.2 Ejecuciones de otros ejemplos

1. Selección de proyectos

1.1. Backtracking

1.1.1. Descripción de la solución

La resolución de este problema mediante backtracking se da mediante la exploración de un árbol binario de estados, que es el espacio de soluciones, en el que se deben encontrar todos los subconjuntos de proyectos cuyo coste acumulado no supere el presupuesto.

Estos son los diferentes elementos de diseño:

- Representación de la solución: Se utiliza una tupla de n componentes donde cada elemento solo puede tomar los valores 1, si se incluye o 0 si no se incluye.
- El algoritmo avanza nivel por nivel, en el nivel i se toma una decisión exclusiva para el proyecto i.
- Función de factibilidad: Antes de explorar la rama a incluir, se comprueba si el coste acumulado actual es mayor que el límite. Si lo supera, se produce una poda para evitar generar descendientes inválidos.
- Como se trata de una exploración en profundidad, al volver hacia atrás, los cambios se deshacen.
- El caso base se alcanza cuando el nivel de profundidad es igual a la cantidad de proyectos, de manera que se ha tomado una decisión para todos ellos y se ha formado un subconjunto válido.

Se han creado diferentes estructuras para resolver el problema:

- Proyectos: Contiene un vector de pares de enteros. Representa los diferentes proyectos que se obtienen, es decir, los candidatos. El primer elemento del pair es el coste, y el segundo el beneficio.
- Solución: Contiene la información requerida para cada nodo así como contadores. Uno de sus atributos es un vector de bool, que representa si un proyecto está seleccionado, el resto son enteros, los cuales representan el coste y beneficio actual, el límite de coste, el contador de podas y de soluciones, y el tamaño, que es igual al número de proyectos.

El código se ha implementado de la siguiente manera:

El algoritmo recibe por parámetro una estructura solución, una estructura proyectos y el nivel actual del árbol.

Caso base:

```
if (indice = tamaño(Proyectos))
        contador_soluciones++;
        Mostrar por pantalla el subconjunto actual junto a su coste y beneficio
        return
end
//Recopilar la información del proyecto actual
coste_actual = coste_proyecto_indice;
beneficio_actual = beneficio_proyecto_indice;
//Incluir el proyecto actual
```

```
if (es_factible) then
        seleccionado = true;
        coste += coste_actual;
        beneficio += beneficio_actual;
        Llamada recursiva sumando uno al nivel
        //Se deshace la decisión
        seleccionado = false;
        coste -= coste_actual;
        beneficio -= beneficio_actual;
else
        //Si no es factible, se poda y se suma al contador
        contador_podas++;
end
//No incluir el proyecto
seleccionado = false;
Llamada recursiva sumando uno al nivel
```

La salida serían los subconjuntos junto con su coste y beneficio y la actualización de los contadores.

Esta es la lógica mediante un caso de uso:
Tenemos los siguientes proyectos

Proyecto	Coste	Beneficio
Proyecto A	4	5
Proyecto B	3	4
Proyecto C	2	3

Con un presupuesto máximo de 6.
La traza de ejecución sería la siguiente:

1. Estado inicial: Coste = Beneficio = 0. Subconjunto vacío
2. PA = 4 <= 6 → Es factible. Coste = 4, Beneficio = 6 Subconjunto {PA}
3. PA + PB = 7 > 6 → No es factible. Sobrepasa el presupuesto, incrementa el contador de podas.
4. Vuelta atrás, prueba PB como no seleccionado.
5. PA + PC = 6 <= 6 → Es factible. Llega al caso base, muestra por pantalla el subconjunto, beneficio y coste y suma uno al contador de soluciones
6. Ahora probaría a no incluir PA, en esta rama todos los subconjuntos son solución.
7. El último nodo por probar es el caso en el que ninguno de los proyectos sea seleccionado, esto también es una solución, por lo que se muestra por pantalla el subconjunto vacío con beneficio y coste 0.

1.1.2. Análisis de eficiencia

En cuanto a la eficiencia teórica, el peor caso consistiría en no realizar ninguna poda y visitar todos los nodos hoja, es decir, la eficiencia es igual a fuerza bruta. Por cada proyecto tenemos dos decisiones posibles, tomarlo o no tomarlo, por lo que el árbol de decisiones es un árbol binario. Es por ello que el número máximo de combinaciones a evaluar es 2^n donde n es el número de proyectos. Por ejemplo, en el caso de uso del apartado anterior, al tener 3 proyectos, el número de combinaciones es 8. Es por ello que se puede concluir con que la eficiencia teórica es de O(2^n).
Respecto a la eficiencia empírica, depende totalmente del comportamiento de la función de factibilidad. Por ejemplo, en el ejemplo anterior, en la práctica, no se hacen 8 comprobaciones, ya que se dan podas. Es por ello que, para este ejercicio, cuanto menor sea el presupuesto y mayor sean los costes de los proyectos, mejor será la eficiencia. Con esto, se puede concluir que empíricamente, el algoritmo es mucho más eficiente que la fuerza bruta.

1.1.3 Ejecución

Para compilar el programa es suficiente con dirigirse al directorio src y escribir el comando g++ seleccion_proyectos_backtracking.cpp -o ../bin/seleccion_proyectos_backtracking.
El ejecutable se encuentra en la carpeta bin. Para ejecutarlo desde el directorio src, se utiliza el comando ./../bin/seleccion_proyectos_backtracking.

Esta es la ejecución para diferentes casos de uso:

Proyecto	Coste	Beneficio
Proyecto A	2	4
Proyecto B	3	5
Proyecto C	5	6

Presupuesto = 8
Resultado de ejecución:
Subconjunto 1: { A B }
Beneficio Total: 9
Coste: 5
Subconjunto 2: { A C }
Beneficio Total: 10
Coste: 7
Subconjunto 3: { A }
Beneficio Total: 4
Coste: 2
Subconjunto 4: { B C }

Beneficio Total: 11
Coste: 8
Subconjunto 5: { B }
Beneficio Total: 5
Coste: 3
Subconjunto 6: { C }
Beneficio Total: 6
Coste: 5
Subconjunto 7: { }
Beneficio Total: 0
Coste: 0
Número soluciones posibles: 7
Número ramas podadas: 1

Proyecto	Coste	Beneficio
Proyecto A	2	4
Proyecto B	3	5
Proyecto C	5	6

Presupuesto = 3
Resultado de ejecución:
Subconjunto 1: { A }
Beneficio Total: 4
Coste: 2
Subconjunto 2: { B }
Beneficio Total: 5
Coste: 3
Subconjunto 3: { }
Beneficio Total: 0
Coste: 0
Número soluciones posibles: 3
Número ramas podadas: 4

Proyecto	Coste	Beneficio
Proyecto A	2	4
Proyecto B	3	5
Proyecto C	5	6

Presupuesto = 12
Resultado de ejecución:
Subconjunto 1: { A B C }
Beneficio Total: 15
Coste: 10
Subconjunto 2: { A B }
Beneficio Total: 9
Coste: 5
Subconjunto 3: { A C }
Beneficio Total: 10
Coste: 7
Subconjunto 4: { A }
Beneficio Total: 4
Coste: 2
Subconjunto 5: { B C }
Beneficio Total: 11
Coste: 8
Subconjunto 6: { B }
Beneficio Total: 5
Coste: 3
Subconjunto 7: { C }
Beneficio Total: 6
Coste: 5
Subconjunto 8: { }
Beneficio Total: 0
Coste: 0
Número soluciones posibles: 8
Número ramas podadas: 0

Proyecto	Coste	Beneficio
Proyecto A	4	5
Proyecto B	5	6
Proyecto C	3	3
Proyecto D	2	2

Presupuesto = 8
Resultado de la ejecución:
Subconjunto 1: { A C }
Beneficio Total: 8
Coste: 7
Subconjunto 2: { A D }

Beneficio Total: 7
Coste: 6
Subconjunto 3: { A }
Beneficio Total: 5
Coste: 4
Subconjunto 4: { B C }
Beneficio Total: 9
Coste: 8
Subconjunto 5: { B D }
Beneficio Total: 8
Coste: 7
Subconjunto 6: { B }
Beneficio Total: 6
Coste: 5
Subconjunto 7: { C D }
Beneficio Total: 5
Coste: 5
Subconjunto 8: { C }
Beneficio Total: 3
Coste: 3
Subconjunto 9: { D }
Beneficio Total: 2
Coste: 2
Subconjunto 10: { }
Beneficio Total: 0
Coste: 0
Número soluciones posibles: 10
Número ramas podadas: 3

1.2 Solución con branch & bound

1.2.1. Descripción de la solución

La resolución de este problema mediante Branch & Bound se basa en la exploración inteligente de un árbol binario de estados. A diferencia del backtracking, no exploramos simplemente en profundidad, sino que priorizamos las ramas más prometedoras para maximizar el beneficio y minimizar el número de nodos visitados.
Estos son los elementos que hemos usado en el diseño del algoritmo :

Representación del estado El estado en cada nodo del árbol se representa mediante una estructura (Nodo) que contiene: Nivel, el proyecto sobre el cual se está tomando una decisión en ese momento del que cogemos Beneficio y Coste: Valores acumulados de las decisiones tomadas hasta el momento. Camino: Un vector de booleanos donde cada elemento toma el valor true (1) si se incluye el proyecto o false (0) si no. Cota: Un valor flotante que representa el beneficio

máximo potencial (estimación optimista) que se podría alcanzar desde ese nodo.
Función de coste (Estrategia de exploración): La estrategia utilizada es Best-First Search (exploración por el mejor nodo). Se utiliza una Cola de Prioridad que ordena los nodos de mayor a menor Cota Superior. De esta forma, el algoritmo siempre expande primero el estado que tiene el mayor techo de beneficio potencial, permitiendo encontrar la solución óptima más rápidamente y mejorar la eficiencia de las podas. Cota superior del beneficio (Upper Bound) para cada nodo, se calcula una cota superior relajando la restricción de integridad (permitiendo fracciones de proyectos). Se suman los beneficios de los proyectos que caben enteros en el presupuesto restante.Para el primer proyecto que ya no cabe, se suma la parte proporcional de su beneficio según el espacio disponible es decir espacio disponible*(benificio(i)/coste(i)).Esto nos da un "techo" matemático: ningún descendiente de ese nodo podrá jamás superar ese valor
Cota inferior inicial (Lower Bound): La cota inferior representa el mejor beneficio real encontrado hasta el momento. Se inicializa en 0 (s.beneficioOptimo = 0). A medida que el algoritmo encuentra combinaciones factibles (donde coste <= presupuesto) cuyo beneficio real supera este valor, la cota inferior se actualiza, permitiendo realizar podas más agresivas en otras ramas. Funciones de Poda el algoritmo utiliza dos tipos de poda para optimizar la búsqueda:Poda por Factibilidad: Antes de generar el hijo que incluye un proyecto, se comprueba si coste + coste_proyecto <= presupuesto. Si se excede, la rama se poda inmediatamente. Poda por Cota (Optimización): Un nodo solo se inserta en la cola de prioridad si su Cota Superior es estrictamente mayor que el mejor beneficio real encontrado hasta el momento (v.cota > s.beneficioOptimo). Si no puede superarlo ni en el mejor de los casos, la rama se descarta. Análisis de las estructuras creadas:
Proyectos: Contiene un vector de pares donde, first es el beneficio y second es el coste. Los proyectos se ordenan inicialmente por eficiencia (ratio beneficio/coste) para garantizar que la cota superior sea lo más ajustada posible.
Solución: Estructura encargada de almacenar los resultados globales: el beneficio óptimo final, la combinación de proyectos que lo logra (seleccionados) y los contadores de rendimiento (nodos generados y podados).

El algoritmo recibe por parámetro una estructura Proyectos (ordenada por eficiencia), el presupuesto máximo y una estructura Solución (pasada por referencia) para almacenar los resultados globales.

Inicialización:
Se crea una Cola de Prioridad de nodos. Se inserta un Nodo Raíz con nivel = -1, coste = 0 y beneficio = 0. Se calcula su Cota Superior inicial.

Bucle Principal (mientras la cola no esté vacía):

Extraer el mejor nodo: Se saca de la cola el nodo u con la mayor cota superior (Estrategia Best-First).

Poda por Cota:
if (u.cota <= beneficioOptimo) then
contador_podas++;
Continuar con el siguiente nodo (saltar exploración)
end

Generar Hijos (Nivel i+1): Se toma una decisión para el siguiente proyecto en el vector ordenado.

A. Rama "Incluir Proyecto" (Hijo Izquierdo):

```
v.coste = u.coste + coste_proyecto_i;
v.beneficio = u.beneficio + beneficio_proyecto_i;
if (v.coste <= presupuesto) then
   if (v.beneficio > beneficioOptimo) then
      beneficioOptimo = v.beneficio;
      guardar_seleccion_actual;
   end
   v.cota = calcularCota(v);
   if (v.cota > beneficioOptimo) then
      Insertar v en la Cola de Prioridad
   else
      contador_podas++;
   end
else
   contador_podas++; // Poda por factibilidad
end
```

B. Rama "No Incluir Proyecto" (Hijo Derecho):

```
v.coste = u.coste;
v.beneficio = u.beneficio;
v.cota = calcularCota(v);
if (v.cota > beneficioOptimo) then
   Insertar v en la Cola de Prioridad
else
   contador_podas++;
end
```

Lógica mediante caso de usoUtilizando los mismos proyectos del ejemplo anterior, pero ordenados por eficiencia (Beneficio/Coste):Proyecto C: B:3, C:2

(Efic: 1.5) Proyecto B: B:4, C:3 (Efic: 1.33) Proyecto A: B:5, C:4 (Efic: 1.25) Presupuesto máximo: 6.
Traza de ejecución simplificada: Estado Inicial: Se calcula la cota del nodo raíz. Caben C y B enteros, y sobra 1 de espacio para una fracción de A. Cota = 3 + 4 + (1 *1.25) = 8.25 .Decisión Proyecto C (el más eficiente):Rama SI incluir C: Coste = 2, Beneficio = 3. Es factible. beneficioOptimo se actualiza a 3. Se calcula su cota (8.25) y se mete en la cola.Rama NO incluir C: Coste = 0, Beneficio = 0. Se calcula su cota (B y fracción de A = 7.75) y se mete en la cola.
Exploración (Best-First): El algoritmo saca de la cola el nodo SI incluir C (Cota 8.25).Decisión Proyecto B: Se genera la rama C + B. Coste = 5, Beneficio = 7. Es factible. beneficioOptimo sube a 7. Su cota sigue siendo 8.25.Decisión Proyecto A:Rama C + B + A: Coste = 9. No es factible (9 > 6). Se incrementa el contador de podas. Rama C + A (sin B): Coste = 6, Beneficio = 8. Es factible. beneficioOptimo sube a 8.Poda final: Al sacar de la cola los nodos restantes (como NO incluir C), su cota (7.75) es menor que el beneficio óptimo actual (8). El algoritmo poda esas ramas directamente sin explorar sus hijos.Salida final:Muestra el beneficio máximo (8) y los contadores. A diferencia del backtracking, no ha tenido que pasar por todas las combinaciones de no elegir C, porque la cota superior habia avisado de que ahí no había nada mejor que el 8 que ya encontró.

1.2.2. Análisis de eficiencia

Eficiencia Teórica
Desde un punto de vista teórico, el Branch & Bound comparte el mismo peor caso que el backtracking y la fuerza bruta. Al ser un problema de decisión binaria (incluir o no incluir cada proyecto), el espacio de estados se representa mediante un árbol binario de profundidad n.
Complejidad temporal: En el peor de los escenarios (cuando las cotas no logran podar ninguna rama), el algoritmo podría llegar a generar $2^(n+1)-1 nodos. Por tanto, su eficiencia teórica pertenece a O(2^n).
Complejidad espacial: A diferencia del backtracking (que usa una pila y consume O(n), el Branch & Bound puede llegar a almacenar una gran cantidad de nodos en la cola de prioridad antes de encontrar la solución óptima, lo que en el peor caso también podría escalar exponencialmente.

Eficiencia Empírica:

En la práctica, el comportamiento del Branch & Bound suele ser significativamente superior a la fuerza bruta y al backtracking gracias a la estrategia Best-First y al uso de cotas superiores e inferiores:
Potencia de la Cota Superior: Al permitir fracciones, el algoritmo estima con mucha precisión el potencial de cada rama. Esto permite descartar subárboles

completos muy pronto si su cota no supera el beneficio óptimo actual (s.beneficioOptimo).
Estrategia de Exploración: Al expandir siempre el nodo más prometedor (el de mayor cota), el algoritmo tiende a encontrar la solución óptima en las primeras etapas de la ejecución. Una vez encontrada una solución real alta, el listón para el resto de los nodos sube, provocando una poda masiva de las ramas restantes.
Factor determinante: La eficiencia empírica mejora drásticamente cuando: Los proyectos están ordenados por eficiencia (beneficio/coste), lo que hace que las cotas sean muy ajustadas.

1.2.3. Ejecución

Para compilar el programa es suficiente con dirigirse al directorio src y escribir el comando g++ seleccion_proyectos_branchandboundb.cpp -o ../bin/seleccion_proyectos_branchandbound.cpp.
El ejecutable se encuentra en la carpeta bin. Para ejecutarlo desde el directorio src, se utiliza el comando ./../bin/seleccion_proyectos_branchandbound.

Esta es la ejecución para diferentes casos de uso:

Proyecto	Coste	Beneficio
Proyecto A	2	4
Proyecto B	3	5
Proyecto C	5	6

Presupuesto = 8
Resultado de ejecución:

Solución Óptima: { B C }
Beneficio Total: 11
Coste Total: 8
Estrategia: Best-First Search
Número nodos generados: 7
Número nodos podados: 2

Proyecto	Coste	Beneficio
Proyecto A	2	4
Proyecto B	3	5
Proyecto C	5	6

Presupuesto = 3
Resultado de ejecución:
Solución Óptima: { B }
Beneficio Total: 5
Coste Total: 3
Estrategia: Best-First Search
Número nodos generados: 5
Número nodos podados: 4

Proyecto	Coste	Beneficio
Proyecto A	2	4
Proyecto B	3	5
Proyecto C	5	6

Presupuesto = 12
Resultado de ejecución:
Solución Óptima: { A B C }
Beneficio Total: 15
Coste Total: 10
Estrategia: Best-First Search
Número nodos generados: 4
Número nodos podados: 0

Proyecto	Coste	Beneficio
Proyecto A	4	5
Proyecto B	5	6
Proyecto C	3	3
Proyecto D	2	2

Presupuesto = 8
Resultado de la ejecución:
Solución Óptima: { B C }
Beneficio Total: 9
Coste Total: 8
Estrategia: Best-First Search
Número nodos generados: 9

Número nodos podados: 6

1.3. Comparativa

1. Diferencias Principales en la Implementación:

Estrategia de Exploración :
Para el bactracking hemos usado Depth-First Search (DFS): Baja hasta el fondo de cada rama por orden. Y para el branch&bound Best-First Search: Usa una priority_queue para explorar primero el nodo más prometedor.

Estructura Recursiva: El backtracking se apoya en la pila de llamadas del sistema. El branch&bound usa una implementación Iterativa: Usa una cola de prioridad y un bucle while.

Orden de los datos :
Para el backtacking Irrelevante para el funcionamiento .Crítico para el branch&bound : Los proyectos deben estar ordenados por eficiencia (Beneficio/Coste).

2. ¿Por qué el Branch & Bound es más eficiente?

El Branch & Bound no es solo más rápido, es más inteligente por tres razones fundamentales:

La Cota Superior : En el código de Backtracking, si una combinación es factible, se explora hasta el final. En Branch & Bound, antes de entrar en una rama, calcula su cota superior. Si esa cota te dice que, en el mejor de los casos, esa rama dará un beneficio de 10 y ya tiene una solución real de 11, el algoritmo ni siquiera entra. Esto elimina miles de nodos muy fácilmente.

Enfoque al Éxito (Best-First): El Backtracking puede perder mucho tiempo explorando ramas malas pero factibles al principio. El Branch & Bound, al usar la cola de prioridad, se dirige primero hacia las soluciones que tienen mayor cota. Esto hace que encuentre el beneficio óptimo muy rápido, lo que a su vez permite podar el resto de las ramas con más facilidad.

3. Diferencia en el Corte de Ramas

En Backtracking: La "poda" solo ocurre cuando S.funcFactibilidad(coste_p) es falso..

En Branch & Bound: Introducimos la poda por optimización. Aunque quepamos en el presupuesto, si la rama es mala comparada con nuestro récord actual (u.cota <= s.beneficioOptimo), la cortamos. Esto convierte al algoritmo en un sistema mucho más eficiente para problemas de gran escala.

Proyecto	Coste	Beneficio
Proyecto A	2	4
Proyecto B	3	5
Proyecto C	5	6

¿Cómo lo hace el Backtracking?
El Backtracking sigue explorando todas las ramas, pero la función S.funcFactibilidad empezará a decir "NO" en cuanto se pase de 8.
Paso 1: Empieza por A (SI), luego B (SI). El coste es 2 + 3 = 5. Ahora intenta añadir C (SI).5 + 5 = 10. Como 10 > 8, PODA POR FACTIBILIDAD. No llega al caso base con los tres.
Paso 2: Vuelve atrás al nivel de C y prueba C (NO).Encuentra {A, B} (Coste 5, Beneficio 9). Subconjunto 1.
Paso 3: Vuelve atrás al nivel de B y prueba B (NO). Luego prueba C (SI).Encuentra {A, C} (Coste 2 + 5 = 7, Beneficio 10). Subconjunto 2.
Paso 4: Vuelve atrás al nivel de A y prueba A (NO). Prueba B (SI) y C (SI). 3 + 5 = 8. Cabe justo. Encuentra {B, C} (Beneficio 11). Subconjunto 3.
Paso 5: Sigue explorando las opciones individuales y el conjunto vacío:Encuentra {A}, {B}, {C} y { }.Resultado Final: Imprime 7 subconjuntos (soluciones posibles) y suma 1 poda (cuando intentó meter los tres proyectos a la vez y no cupieron).

Proyecto	Coste	Beneficio
Proyecto A	2	4
Proyecto B	3	5
Proyecto C	5	6

Presupuesto = 8
Resultado de ejecución:

Paso 1 (Raíz): Calcula la cota superior usando relajación (fracciones).Caben A (2) y B (3). Sobran 3 de espacio para una fracción de C.Cota: 4 + 5 + (3/5 * 6) = 12.6. Se mete en la cola.
Paso 2: Genera los hijos del proyecto más eficiente (A).Rama A-SI: Coste 2, Ben 4. Cota: 12.6 (porque aún aspira a meter B y trozo de C).Rama A-NO: Coste 0, Ben 0. Cota: 11 (lo que darían B + C enteros, que caben en 8).Cola: {A-SI (12.6), A-NO (11)}.
Paso 3 (Best-First): La cola de prioridad extrae el más prometedor: A-SI (Cota 12.6).
Paso 4: Genera hijos para el proyecto B (viniendo de A-SI).Rama A-SI + B-SI: Coste 5, Beneficio 9. Cota: 12.6. (Récord provisional: 9).Rama A-SI + B-NO: Coste 2, Beneficio 4. Cota: 10 (4 de A + 6 de C que cabe entero).Cola: {A-SI+B-SI (12.6), A-NO (11), A-SI+B-NO (10)}.

Paso 5: Extrae el mejor de la cola: A-SI + B-SI (Cota 12.6). Genera hijos para C.Rama A-SI + B-SI + C-SI: Coste 5+5=10. PODA POR FACTIBILIDAD (10 > 8).Rama A-SI + B-SI + C-NO: Beneficio 9. Es solución real.Cola: {A-NO (11), A-SI+B-NO (10)}.
Paso 6: Extrae el siguiente mejor: A-NO (Cota 11). Genera hijos para B.Hijo A-NO + B-SI: Coste 3, Ben 5. Cota: 11 (aspira a meter a C entero).Hijo A-NO + B-NO: Coste 0, Ben 0. Cota: 6.Cola: {A-NO+B-SI (11), A-SI+B-NO (10), A-NO+B-NO (6)}.
Paso 7: Extrae A-NO + B-SI (11). Genera hijos para C.Rama A-NO + B-SI + C-SI: Coste 3+5=8, Beneficio 5+6=11.Es factible y tenemos un nuevo récord real de 11.
Paso 8 (Poda final): Mira lo que queda en la cola:Nodo A-SI + B-NO (Cota 10). Como 10 < 11, SE PODA.Nodo A-NO + B-NO (Cota 6). Como 6 < 11, SE PODA.

2. Coloreo de grafos

2.1. Backtracking

2.1.1. Descripción de la solución

La resolución de este problema mediante backtracking se da mediante la exploración de un árbol de estados que representa el espacio de soluciones, en el que se debe encontrar una asignación de colores a los vértices del grafo tal que ningún par de vértices adyacentes comparta el mismo color, usando como máximo K colores.

Estos son los diferentes elementos de diseño:

Representación de la solución: Se utiliza una tupla de n componentes donde cada elemento puede tomar un valor entero entre 0 y K-1, representando el color asignado a ese vértice, o -1 si todavía no tiene color asignado. El algoritmo avanza nivel por nivel; en el nivel i se toma una decisión exclusiva para el vértice i, eligiendo qué color se le asigna.

Función de factibilidad: Antes de explorar la rama correspondiente a un color, se comprueba si asignar ese color al vértice actual entra en conflicto con algún vecino ya coloreado.Para ello se recorre la lista de adyacencia del vértice y se verifica que ningún vecino tenga ese mismo color. Si se detecta conflicto, se hace una poda para evitar generar descendientes inválidos.

Como se trata de una exploración en profundidad, al volver hacia atrás el color asignado se deshace, poniendo de nuevo el vértice a -1.

El caso base se alcanza cuando el nivel de profundidad es igual al número de vértices del grafo, de manera que se ha tomado una decisión de color para todos ellos y se ha formado un coloreo válido completo.

Se han utilizado las siguientes estructuras para resolver el problema:

Graph: Contiene el número de vértices y una lista de los adyacentes representada como un vector de vectores de enteros.Cada posición i almacena los vecinos del vértice i.

ColoringResult: Contiene la información para cada nodo así como unos contadores.Uno de sus atributos es un vector de enteros colors, que representa el color asignado a cada vértice (-1 si el valor es sin asignar) el resto son enteros que representan el número de colores utilizados, el contador de nodos generados, el contador de nodos podados y el máximo de nodos vivos simultáneamente en la pila de recursión.

El pseudocódigo es este:
Caso base:

```
si (vertex == num_vertices)
  success = true
  colors_used = contar_colores_distintos(colors)
  return true
fin

// Actualizar la profundidad máxima en la pila //
max_live_nodes = max(max_live_nodes, vertex + 1)

// Probar cada color posible para el vértice actual //
para color desde 0 hasta max_colors - 1:
   nodes_generated++

   si (es_factible(vertex, color, colors, grafo)) entonces
      colors[vertex] = color
      si (llamada recursiva con vertex+1 devuelve true)
         return true
      fin
      // Se deshace la decisión //
      colors[vertex] = -1
   sino
```

```
      // No es factible: se poda y se suma al contador //
      nodes_pruned++
   fin
fin
return false
```

Ahora vamos a poner de ejemplo con un grafo de 4 vértices y 3 colores:
v0 tiene de vecinos a v1,v2,v3

v1 tiene de vecinos v0,v2

v2 tiene de vecinos v0,v1

v3 tiene de vecinos a v0

La traza de ejecución sería la siguiente:

Estado inicial**:** colors = [-1, -1, -1, -1]. Ningún vértice tiene un color asignado.

v0 = color 0 : is_color_valid: sin vecinos coloreados = válido, colors = [0, -1, -1, -1]

nodes_generated = 1

v1 = color 0 : is_color_valid: v0 es vecino y tiene color 0 = conflicto

Poda : nodes_generated = 2, nodes_pruned = 1

v1 = color 1: is_color_valid: v0 tiene color 0, ningún vecino tiene color 1 = válido,

colors = [0, 1, -1,-1]

nodes_generated = 3.

v2 = color 0 : is_color_valid: v0 es vecino y tiene color 0 = conflicto

Poda: nodes_generated = 4, nodes_pruned = 2

v2 = color 1: is_color_valid: v1 es vecino y tiene color 1 = conflicto

Poda, nodes_generated = 5, nodes_pruned = 3

v2 = color 2 : is_color_valid: v0 tiene 0, v1 tiene 1, ninguno tiene 2 = válido

colors = [0, 1, 2, -1], nodes_generated = 6.

v3 = color 0: is_color_valid: v0 es vecino y tiene color 0 = conflicto

Poda, nodes_generated = 7, nodes_pruned = 4

v3 = color 1 : is_color_valid: v0 tiene 0, ningún vecino tiene 1 = válido

colors = [0, 1, 2, 1], nodes_generated = 8

vertex == 4 == num_vertices : caso base alcanzado.

success = true, colors_used = 3, max_live_nodes = 4.

Solución encontrada: v0 = 0, v1 = 1, v2 = 2, v3 = 1

2.1.2.Análisis de eficiencia

En cuanto a la eficiencia teórica, el peor caso consistiría en no realizar ninguna poda y visitar todos los nodos del árbol de búsqueda, es decir, la eficiencia es igual a fuerza bruta. Para cada vértice tenemos K decisiones posibles, donde K es el número máximo de colores permitidos, por lo que el árbol de decisiones es un árbol K-ario de profundidad n siendo la n el número de vértices y por ello que el número máximo de nodos a evaluar es K^n, donde n es el número de vértices y K el número de colores. Por ejemplo, en el caso de uso del apartado anterior, al tener 4 vértices y K=3 colores, el número máximo de nodos sería 3^4 = 81. Es por ello que se puede concluir con que la eficiencia teórica en el peor caso es de O(K^n)

Sin embargo, en la práctica la función de factibilidad is_color_valid poda de forma agresiva el árbol de búsqueda. Cada vez que se detecta un problema con un vecino ya coloreado, se descarta inmediatamente ese color y todos los caminos que dependen de él.

Por tanto dado que el algoritmo devuelve la primera solución válida encontrada sin explorar el árbol completo, en el caso promedio la eficiencia real es considerablemente mejor que el peor caso teórico, pero como el problema de coloreo de grafos es NP-completo, no existe ningún algoritmo de tiempo polinomial conocido que lo resuelva en el caso general, y el peor caso exponencial es inevitable para las instancias arbitrarias.

2.1.3.Ejecución

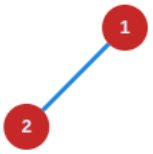
Para la ejecución de map_coloring_example nos vamos a ayudar con el árbol de ejecución que se vea claro que ramas vamos a podar y por cuales seguimos. En este caso, como

estamos en backtracking, el algoritmo buscará cumplir con el número máximo de colores que se le han introducido como argumento, y devolverá la primera solución válida en caso de que la haya. Además, para este ejemplo usaremos los colores rojo, verde y azul en dicho orden como colores posibles a elegir, y que la exploración del árbol la marcaré con aristas dirigidas.

En el nodo raíz el algoritmo escoge el color **rojo** ya que no tiene ninguna restricción.



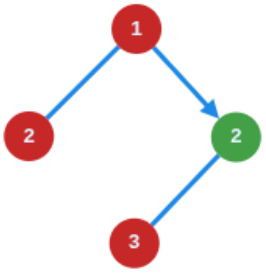
Ahora el algoritmo explora el nodo 2 y comienza a probar los colores posibles. Como este nodo está conectado al 1 que es rojo, **poda** el árbol que partiría de ahí.



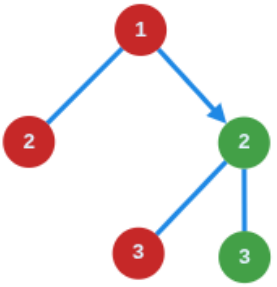
Ahora el algoritmo prueba para el siguiente color que es el **verde**, que como nadie a sus alrededores lo tiene asignado lo escoge y continua por esa rama.



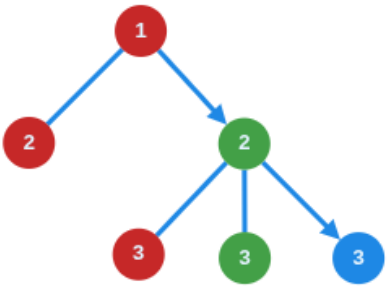
Ahora el algoritmo explora el nodo 3 y de nuevo prueba con los colores posibles. Debido a que este nodo está conectado al 1 el algoritmo **poda** esta rama y comprueba el siguiente color.



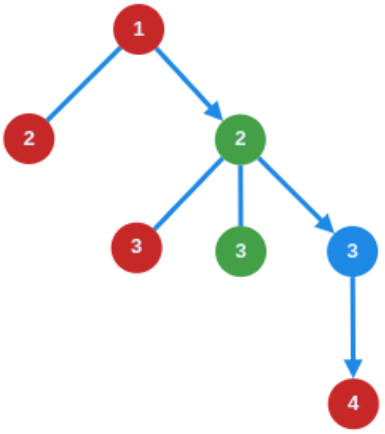
El algoritmo prueba con el color verde, pero como el nodo 3 está conectado también con el 2, **poda** esta rama y continúa en busca del color.



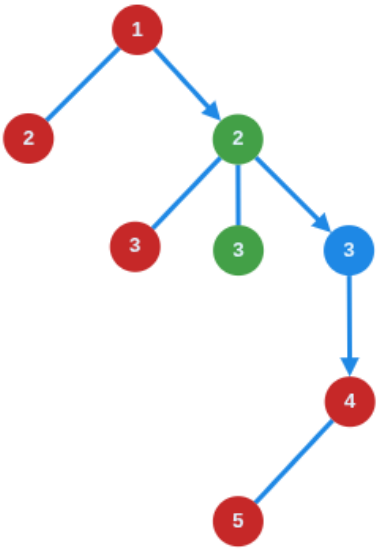
Ahora, el algoritmo comprueba si el color **azul** es posible, y como ningún vecino lo tiene asignado, continúa por esa rama.



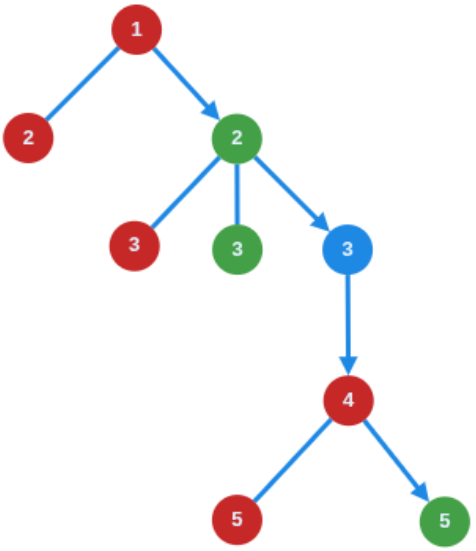
Inmediatamente, el algoritmo explora el nodo 4 y prueba con el color **rojo**, que como ningún vecino lo tiene asignado, lo pone con éxito y continúa por esa rama.

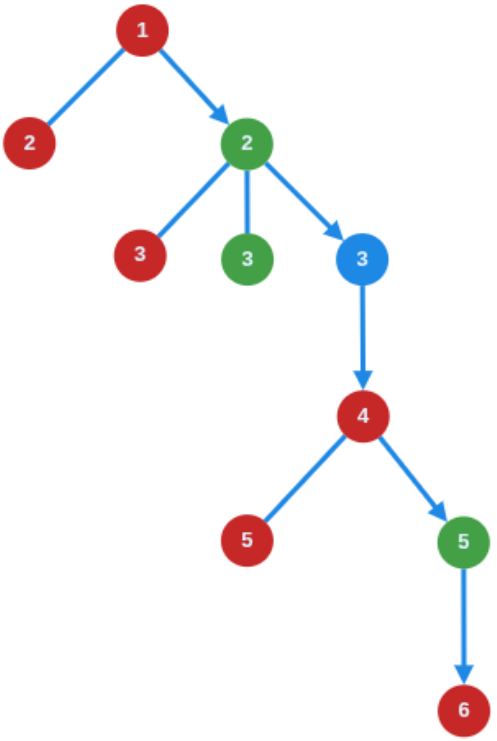


Ahora el algoritmo entra en la exploración del nodo 5, y como este está conectado con el nodo 4, no guarda el rojo y **poda** la rama para seguir buscando un color válido.



En este caso el algoritmo prueba con el color **verde**, y como no lo tiene ningún vecino, le asigna este color y sigue explorando esta rama.





Por último, el algoritmo comienza con la exploración del sexto nodo y como este está únicamente conectado con el nodo 5 que es verde, comprueba el color **rojo** y se lo asigna, encontrando la primera solución válida y **terminando** las **recursiones en cascada**.

2.2. Branch and Bound

2.2.1. Descripción de la solución

La resolución de este problema mediante Branch & Bound se da mediante la exploración de un árbol de estados que representa el espacio de soluciones, en el que se debe encontrar la asignación de colores a los vértices del grafo que minimice el número de colores utilizados, cumpliendo que ningún par de vértices adyacentes comparta color.

Estos son los diferentes elementos de diseño:

Representación de la solución: Se utiliza una tupla de n componentes donde cada elemento puede tomar un valor entero entre 0 y cota_superior-1, representando el color asignado a ese vértice, o -1 si todavía no tiene color asignado. El algoritmo avanza nivel por nivel; en el nivel i se toma una decisión exclusiva para el vértice i, eligiendo qué color se le asigna.

Función de coste: El coste de una solución completa es el número de colores distintos utilizados en la asignación, calculado mediante count_used_colors , y el objetivo es minimizar este valor.

Cota superior inicial: Antes de iniciar la búsqueda se ejecuta greedy_coloring, que ordena los vértices por grado descendente y asigna a cada uno el menor color que no tenga problema con sus vecinos ya coloreados.

El número de colores que usa este algoritmo voraz es la cota superior inicial: sabemos que existe una solución con ese número de colores, por lo que cualquier rama que no pueda mejorarla se va descartando.

Cota inferior: En cada nodo parcial, la cota inferior es el número de colores distintos ya comprometidos en la asignación parcial actual, calculado con count_used_colors (colores_actuales). Esa cantidad no puede decrecer mientras avanza el árbol, ya que los colores ya asignados no se retiran.

Función de poda: Se producen dos tipos de poda, la primera ocurre cuando cota_inferior >= cota_superior: el nodo actual ya usa tantos colores como la mejor solución conocida, por lo que cualquier extensión de esta rama solo podría igualarla o empeorarla, nunca mejorarla. La segunda ocurre dentro del bucle de colores: solo se prueban colores de 0 a cota_superior - 1, evitando explorar coloreos que introduzcan más colores que la mejor solución ya encontrada. Ademas , si is_color_valid devuelve false, se poda esa rama por conflicto.

Como se trata de una exploración en profundidad, al volver hacia atrás el color asignado se deshace, poniendo el vértice de nuevo a -1.

El caso base se alcanza cuando el nivel de profundidad es igual al número de vértices. En ese momento se ha encontrado un coloreo completo válido con menos colores que la mejor solución conocida, por lo que se actualiza cota_superior , se guarda el coloreo y se continúa buscando si existe aún una solución con menos colores.

Se han utilizado las siguientes estructuras para resolver el problema:

Graph: Contiene el número de vértices y una lista de los adyacentes representada como un vector de vectores de enteros.

ColoringResult: Contiene el coloreo óptimo encontrado en colors, el número de colores utilizados en colors_used, los contadores nodes_generated, nodes_pruned y max_live_nodes, y con success indica si se encontró solución.

El pseudocódigo es este:

Caso base:

```
si (vertex == num_vertices)
    colors = colores_actuales
    colors_used = cota_inferior
    success = true
    cota_superior = cota_inferior  //actualizamos para futuras podas//
    return
```

```
fin


// Poda por cota //
cota_inferior = count_used_colors(colores_actuales)
si (cota_inferior >= cota_superior)
   nodes_pruned++
   return
fin


// Probar cada color dentro del rango útil //
para color desde 0 hasta cota_superior - 1:
   nodes_generated++
   si (is_color_valid(vertex, color, colores_actuales, grafo)) entonces
      colores_actuales[vertex] = color
      llamada recursiva con vertex+1
      colores_actuales[vertex] = -1
   sino
      nodes_pruned++
   fin
fin
```

Ahora vamos a poner de ejemplo con un grafo 4 vértices con las aristas v0-v1, v0-v2, v0-v3, v1-v2.

Estado inicial: greedy_coloring asigna v0 = C0, v1 = C1, v2 = C2, v3 = C1

colors_used = 3

cota_superior = 3

colores_actuales = [-1,-1,-1,-1]

vertex=0: cota_inferior 0 < 3

Probamos el color 0 (C0)

is_color_valid = true

colores_actuales = [0,-1,-1,-1]

vertex=1:cota_inferior 1 < 3.

Probamos C0 = false (v0 vecino)

nodes_pruned++

Probamos C1 = true

colores_actuales = [0,1,-1,-1]

vertex=2:cota_inferior 2 < 3

Probamos C0 = false (v0 vecino)

Probamos C1 = false (v1 vecino)

Probamos C2 = true

colores_actuales = [0,1,2,-1]

vertex=3: cota_inferior 3 <= 3

Hacemos poda por cota ya que esta rama no puede mejorar la solución actual.

Vamos para atrás hasta vertex =2: Probamos otros colores para v2, todos fallan por conflicto o por la cota. vamos para atrás hasta vertex=1,

probamos C2 para v1 = valid.

colores_actuales = [0,2,-1,-1]

vertex=2 con v1=C2: Probamos C0 = false (v0)

Probamos C1 = true.

colores_actuales = [0,2,1,-1]

vertex=3: cota_inferior 3 <= 3.

Por poda seguimos explorando hasta agotar ramas con cota_superior = 3 ya que ninguna mejora a 2 colores es posible.

Resultado final: colors_used = 3,es la solución óptima encontrada:, es el número de colores del grafo.

2.2.2.Análisis de eficiencia teórica

En cuanto a la eficiencia teórica, el peor caso del Branch & Bound coincide con el del backtracking: si las cotas no consiguen eliminar ninguna rama, el algoritmo visita todos los nodos del árbol de búsqueda, siendo equivalente a fuerza bruta. Para cada vértice se prueban como máximo cota_superior colores, donde cota_superior es el número de colores de la solución voraz inicial, por lo que el árbol de decisiones es un árbol de aridad cota_superior y profundidad n. El número máximo de nodos a evaluar es por tanto cota_superior^n, donde n es el número de vértices. Por ejemplo en el caso de uso del apartado anterior, al tener 4 vértices y una cota superior inicial de 3 colores, el número máximo de nodos sería 3^4 = 81. Es por ello que se puede concluir con que la eficiencia teórica en el peor caso es de O(K^n), donde K es el valor de la cota superior inicial.

Sin embargo, en la práctica el Branch & Bound introduce dos mecanismos de poda que lo hacen considerablemente más eficiente que el backtracking puro. El primero es la poda por factibilidad mediante is_color_valid, que descarta colores en conflicto con vecinos que ya están coloreados. El segundo, y que es exclusivo del Branch & Bound, es la poda por cota: cuando el número de colores ya comprometidos en la asignación parcial es igual o mayor que la mejor solución conocida hasta el momento, se descarta toda esa rama sin explorarla, ya que no puede producir una solución mejor. Además, cada vez que se encuentra una solución completa que mejora la cota superior, el rango de colores a probar en el bucle se reduce automáticamente, haciendo el árbol efectivamente más estrecho a medida que avanza la búsqueda.

Por tanto, aunque el peor caso teórico es el mismo que el del backtracking, en la práctica el Branch & Bound genera y poda menos nodos, especialmente en grafos donde la solución voraz inicial es cercana al óptimo. No obstante, como el problema de coloreo de grafos es NP-completo, no existe ningún algoritmo de tiempo polinomial conocido que lo resuelva en el caso general, y el peor caso exponencial es inevitable para instancias arbitrarias.

2.2.3.Ejecución

Para la ejecución de map_coloring_example, al igual que antes, nos vamos a ayudar con el árbol de ejecución que se vea claro que ramas vamos a podar y por cuales seguimos. En este caso, como estamos en branch and bound, el algoritmo buscará el mínimo número de

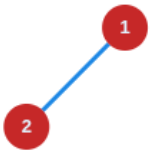
colores con el que se pueda pintar el grafo. Además, para este ejemplo usaremos los colores rojo, verde y azul en dicho orden como colores posibles a elegir, y que la exploración del árbol la marcaré con aristas dirigidas(igual que lo hemos utilizado en backtracking).

Como primer paso del algoritmo, este ejecuta una búsqueda greedy para encontrar rápidamente la cota superior. Al finalizar, el algoritmo nos devuelve que el número de colores que ha usado es de 3 por lo que nuestro algoritmo va a tratar de reducir ese número en caso de que se pueda.

En el nodo raíz el algoritmo escoge el color **rojo** ya que no tiene ninguna restricción.



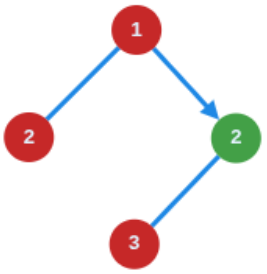
Ahora el algoritmo explora el nodo 2 y comienza a probar los colores posibles. Como este nodo está conectado al 1 que es rojo, **poda** el árbol que partiría de ahí.



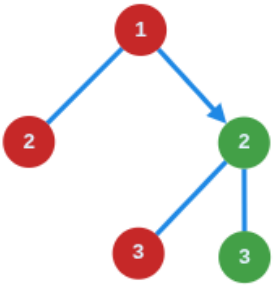
Ahora el algoritmo prueba para el siguiente color que es el **verde**, que como nadie a sus alrededores lo tiene asignado lo escoge y continua por esa rama.



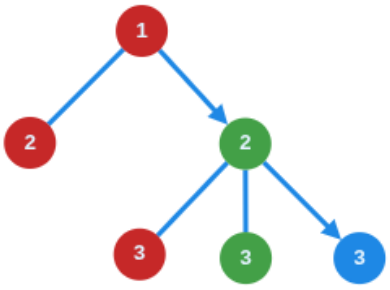
Ahora el algoritmo explora el nodo 3 y de nuevo prueba con los colores posibles. Debido a que este nodo está conectado al 1 el algoritmo **poda** esta rama y comprueba el siguiente color.



El algoritmo prueba con el color verde, pero como el nodo 3 está conectado también con el 2, **poda** esta rama y continúa en busca del color.



Ahora, el algoritmo comprueba si el color **azul** es posible, y como ningún vecino lo tiene asignado, se lo adjudica al nodo.



Ahora se da el caso que hace especial a este algoritmo. Como el grafo nos ha obligado a utilizar tres colores distintos, nuestra cota inferior actual, que es el número de colores que tenemos, el algoritmo poda la rama del 2 verde completa ya que no se puede mejorar el caso del algoritmo greedy.
A continuación el algoritmo probará con el 2 azul pero ocurrirá exactamente lo mismo que antes, podando así la rama del 1 rojo completa y volviendo a repetir estos mismos pasos para el 1 verde y azul.
Una vez completados los tres árboles que se crearán con los tres 1 de distinto color, el algoritmo se da cuenta de que no se puede superar la solución greedy por lo que la devuelve directamente.

2.3. Comparativa

2.3.1 Ejecución map_coloring_example

	Backtracking	Branch and Bound
Nodos generados	10	30
Nodos podados	4	21
Tiempo de ejecución (µs)	2	11
Solución alcanzada	3	3

Como podemos comprobar, en backtracking el algoritmo genera muy pocos nodos y es extremadamente rápido y con respecto a la representación visual podemos comprobar que los datos confirman exactamente nuestro supuesto de ejecución, siendo así que en cuanto encuentra la primera solución que cumple con la restricción impuesta el algoritmo finaliza.

Respecto a branch and bound podemos ver que es un poco más lento que el de backtracking y que genera el triple de nodos pero era de esperar ya que el de branch and bound comprueba absolutamente todas las combinaciones posibles para asegurarse de que no hay ninguna opción mejor. Además, viendo nuestra representación gráfica también nos concuerda perfectamente ya que como habíamos previsto, el algoritmo vuelve a comenzar desde la raíz probando con otros colores para comprobar que la solución sea óptima.

2.3.2 Ejecuciones de otros ejemplos

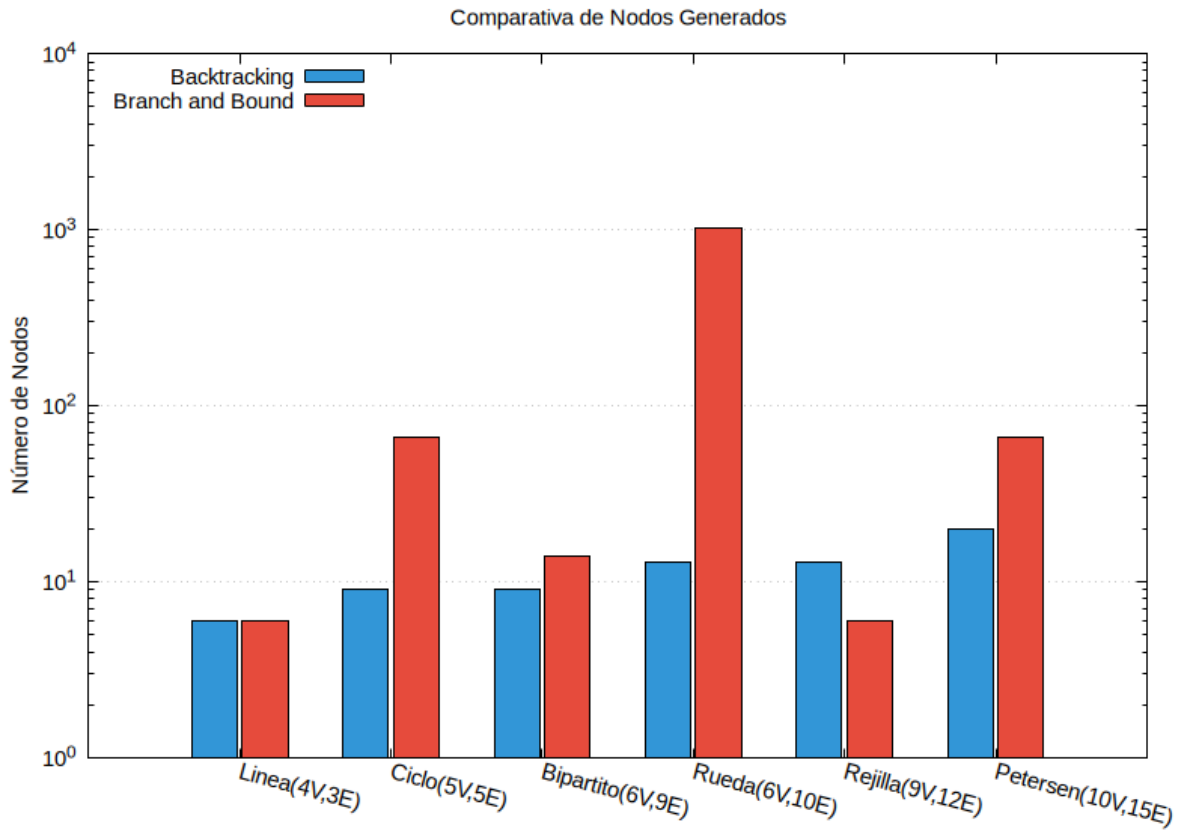
Para evaluar el comportamiento de los algoritmos de Backtracking y de Branch and Bound hemos diseñado diferentes entradas variando el número de vértices y de aristas para hacer una comparación.

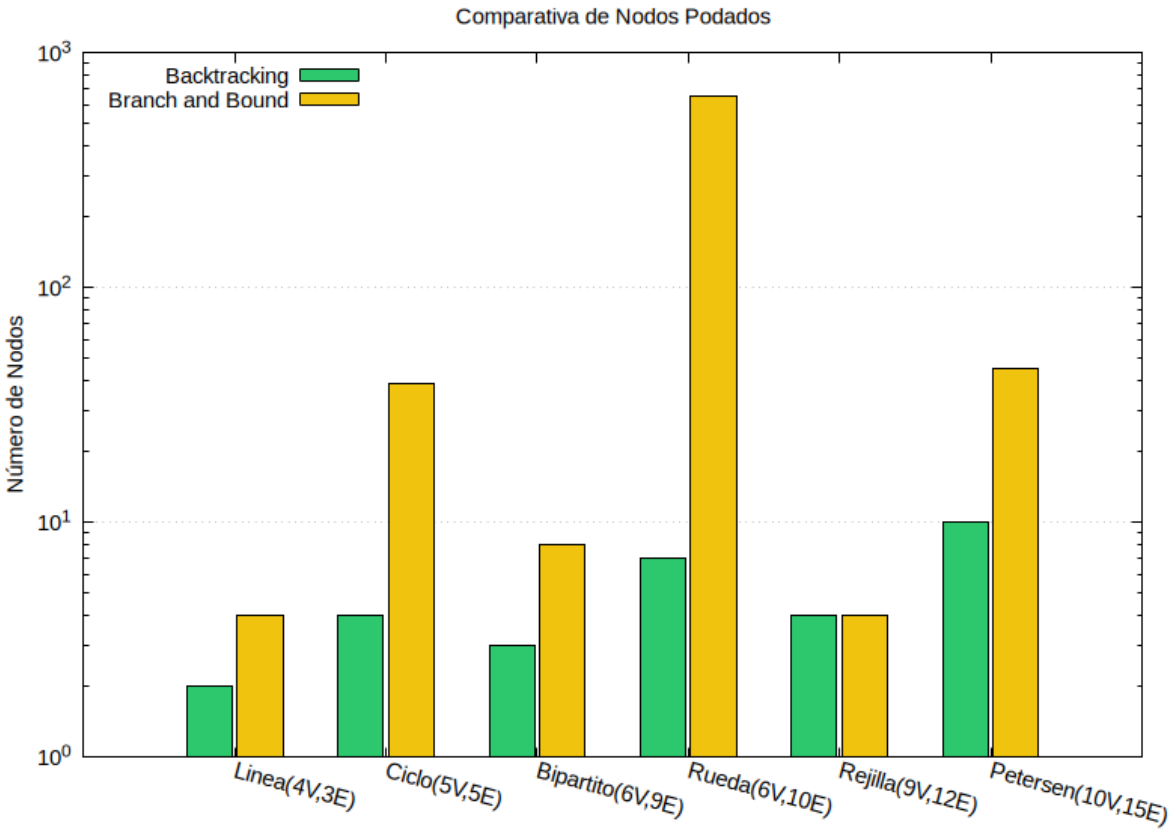
Nodos generados y podados

Como podemos observar en las gráficas que hay a continuación, en prácticamente todos los escenarios, Backtracking genera un número considerablemente menor de nodos en comparación con Branch and Bound. Por ejemplo en “Ciclo” BT generó 9 nodos mientras que BB generó 66. Además, esta diferencia se hace mucho más notable en grafos de mayor densidad y número cromático como por ejemplo “Rueda”, donde BT exploró solo 13 nodos mientras que BB 1028.

Esto ocurre debido a que a BT ya se le establece un límite de colores de antemano, siendo así que, como ya he explicado anteriormente, el algoritmo se detiene una vez encuentra una primera solución que cumple el requisito, mientras que BB continúa explorando todas las posibilidades en busca de la combinación óptima.

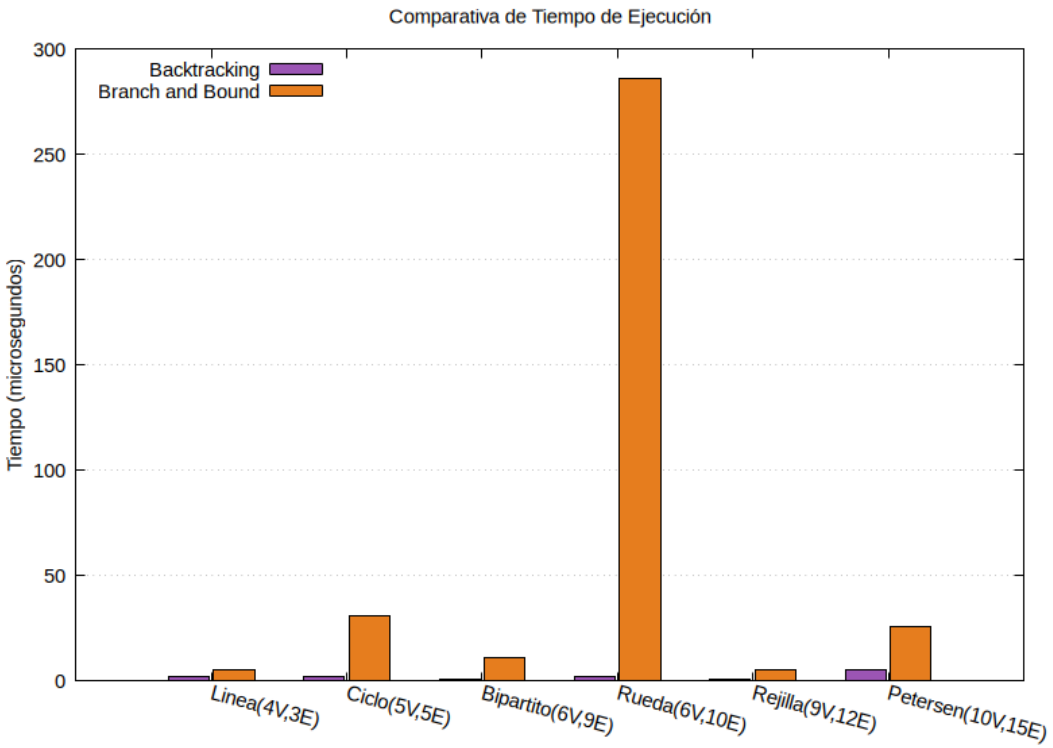
El único caso en el que ha ocurrido la situación inversa ha sido en “Rejilla”, ya que teniendo la solución voraz de 2 colores, BB poda las ramas prácticamente de inmediato.





Tiempos de ejecución

Como podemos observar, la carga de cómputo se refleja directamente en los tiempos de ejecución, siendo así que en ni un sólo ejemplo de los que hemos hecho el tiempo de ejecución de BB es menor que el de BT, y siendo así que Backtracking mantiene un tiempo casi constante oscilando entre 1 y 5 microsegundos.



Conclusión

Para finalizar, hemos deducido que si lo único que necesitas es comprobar si un número cromático es válido para un grafo es infinitamente mejor BT, mientras que si lo que buscas es averiguar la solución más óptima para este problema lo más recomendable es BB ya que a pesar de que su tiempo de ejecución puede ser bastante mayor, siempre te devolverá la solución óptima.